

Codeforces 449B

Jzzhu and Cities

Dijkstra com análise de arestas redundantes

Grupo F

Integrantes: João Vitor Silva • Antonio Davi • Pablo Dornelles

Orientador: Prof. Me Ricardo Carubbi

Linguagem: Python 3 • Plataforma: Codeforces

1. O problema

Enunciado

n cidades, sendo a cidade **1** a capital.

m estradas bidirecionais com pesos x .

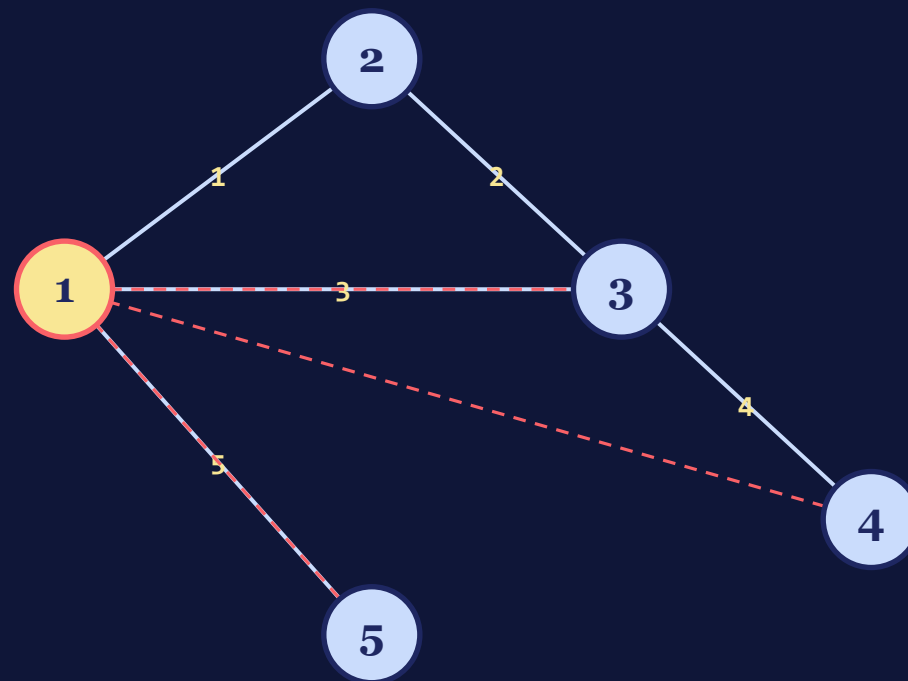
k rotas de trem ligando a capital a uma cidade s com custo y .

Objetivo

Fechar o maior número possível de rotas de trem sem alterar a distância mínima da capital a nenhuma cidade.

Limites: $n \leq 10^5$ • $m \leq 3 \cdot 10^5$ • $k \leq 10^5$ •
pesos até 10^9

Exemplo 1



— estrada

- - - trem (da capital)

2. Modelagem como grafo

1

Vértices

As n cidades. A capital é o vértice 1, origem do Dijkstra.

2

Arestas — estradas

Cada estrada (u, v, x) vira aresta bidirecional de peso x .

3

Arestas — trens

Cada trem (s, y) vira uma aresta extra ligando $1 \rightarrow s$ com peso y , marcada como tipo "trem".

4

Pesos

Todos não negativos ($x, y \geq 1$) $\Rightarrow$ Dijkstra é aplicável.

5

Representação

Lista de adjacência. Cada item: (vizinho, peso, é_trem).

3. Estratégia: Dijkstra + marcação de alternativa

Uma única execução de Dijkstra a partir da capital, sobre estradas + trens.

Relaxamento adaptado

ao relaxar $u \rightarrow v$ com custo nd :

```
se  $nd < dist[v]$ :  
     $dist[v] = nd$   
     $via\_estrada[v] = (não\ é\ trem)$ 
```

```
senão se  $nd == dist[v]$ :  
    se a aresta é estrada:  
         $via\_estrada[v] = True$ 
```

Contagem final, por trem (s, y)

$y > dist[s]$

→ fecha

Trem mais caro que o caminho mínimo: inútil.

$y = dist[s]$ e $via_estrada[s]$

→ fecha

Existe alternativa só por estradas.

$y = dist[s]$ e $\neg via_estrada[s]$

→ mantém 1 / fecha demais

Trem é a única forma de alcançar o ótimo.

$y < dist[s]$

→ impossível

O próprio trem já estava no Dijkstra.

4. Por que funciona

Observação-chave: todo trem sai da capital. Portanto, em qualquer caminho mínimo, um trem só pode aparecer como **a primeira (e única) aresta**. Para vértices que não são destino direto de trem, o problema é Dijkstra padrão — só precisamos decidir o destino de cada trem.

Trem inútil

$y > \text{dist}[s]$

O melhor caminho até s já é menor que o trem; fechar não muda nada.

Alternativa por estrada

$y = \text{dist}[s]$ e $\text{via_estrada}[s]$

Há um caminho ótimo cujo último passo é estrada — manter o trem é redundante.

Trem essencial

$y = \text{dist}[s]$ e $\neg \text{via_estrada}[s]$

Sem trem, $\text{dist}[s]$ aumentaria. Mantém-se exatamente um; duplicados são fechados.

5. Complexidade

Tempo

$$O((V + E) \cdot \log V)$$

$V = n$ (vértices)
 $E = 2m + k$ (estradas dos dois lados + trens)
 $+ O(k)$ contagem final sobre os trens

Fila de prioridade: *heapq* (heap binário).

Memória

$$O(V + E)$$

Listas de adjacência + vetores *dist*, *via_estrada* e a *heap*.

No problema

$n \leq 10^5$ $m \leq 3 \cdot 10^5$ $k \leq 10^5$
 $\Rightarrow \sim 7 \cdot 10^5$ arestas, $\sim 6 \cdot 10^6$ operações de *heap*.

Submissão real: **1625 ms** / **180 MB** (limite 2 s / 256 MB).

6. Casos especiais

Pesos grandes (até 10^9)

Distâncias podem chegar a $\sim 10^{14}$. Python usa inteiros arbitrários, não há overflow.

Múltiplos trens para a mesma cidade

Agrupamos por destino. Mantemos no máximo um quando estritamente necessário; os demais empatados são fechados.

Arestas paralelas entre o mesmo par

Tratadas naturalmente — cada uma vira uma entrada própria na lista de adjacência.

Empate trem $\times$ estrada ($y = \text{dist}[s]$)

A marcação `via_estrada[s]` garante reconhecer alternativa por estrada e fechar o trem redundante.

Trem que melhora a distância

É só mais uma aresta no Dijkstra. `dist[s]` é atualizado pelo trem e `via_estrada[s]` permanece falso $\Rightarrow$ trem mantido.

Conectividade garantida

O enunciado garante que toda cidade alcança a capital, então nenhum vértice fica com distância infinita.

Resultado

Accepted

#376408159

ID da submissão

1625 ms

tempo (limite 2 s)

180 MB

memória (limite 256 MB)

Em resumo

- Modelagem unificada: trens são apenas arestas extras da capital.
- Uma única execução de Dijkstra com a marcação `via_estrada` já resolve.
- Contagem direta sobre os trens, agrupados por destino, dá a resposta.